A Text Mining and Classification Approach for Analyzing Architecture Decision Records: Evidence from Open-Source Repositories

Anonymous Author(s)

ABSTRACT

Architectural decision records (ADRs) have become a popular lightweight approach for documenting architectural knowledge alongside code in software projects. Despite their growing adoption and perceived benefits, there is limited empirical understanding of the architectural concerns captured in ADRs and how well they align with established bodies of architectural knowledge. This study conducts an analysis of ADRs from ≈ 300 open-source repositories, applies data mining techniques to uncover the types of concerns actually captured, assesses their relations with existing classifications of design decisions and quality-attribute aspects, and checks the adherence of the ADR contents (e.g., drivers, rationale, analysis of alternatives) to the well-known MADR template. To automate (most of) the analysis, we define a pipeline combining topic modeling and LLM-based classification that can reveal patterns, insights, and mismatches in a large collection of ADRs. Our findings show that, while ADRs capture a range of decisions and architectural concerns, there are notable discrepancies between the documented content and theoretical expectations. Certain concerns and quality attributes are underrepresented, and there are also inconsistencies in how the MADR template is followed. Furthermore, we discuss the challenge of aligning LLM-based classification with human assessments of ADRs. This work provides architects with valuable information to reflect on how ADRs are and should be used to preserve architectural knowledge more effectively in projects.

KEYWORDS

architectural knowledge, architecture decision records, documentation mining, topic modeling, LLMs

ACM Reference Format:

Anonymous Author(s). 2026. A Text Mining and Classification Approach for Analyzing Architecture Decision Records: Evidence from Open-Source Repositories. In *Proceedings of The 30th International Conference on Evaluation and Assessment in Software Engineering (EASE 2026).* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Capturing design decisions for software architectures has been regarded as a valuable practice in software projects. The main goal is to document the key decisions and rationale that guide system evolution and maintenance so that this knowledge can be accessible to different project stakeholders. Architecture knowledge (AK) refers to a collective understanding of the design goals, constraints, rationale, alternatives, and tradeoffs involved in the design of a system. To avoid knowledge vaporization, a number of tools and techniques for capturing and sharing AK have been proposed [1].

A popular and lightweight approach for AK management is to use *architecture decision records* [2, 3]. An architecture decision record (ADR) provides a structured template for recording the main context, forces, alternatives, pros and cons, and consequences (among other elements) of design decisions. ADRs can be specified using a textual format (e.g., Markdown) and a simple template, such as MADR[1], enabling architects and developers to seamlessly capture the design rationale in the development workflow[2].

Although ADRs appear as an attractive documentation approach with many advocates, there is little empirical evidence on how ADRs are used by practitioners. In particular, we are interested in the kind of architectural concerns that practitioners often choose to document and how those concerns are articulated in their ADRs. Despite single-case experience reports, to our knowledge, there is only one study [4] that analyzed a collection of open-source projects using ADRs. This study revealed interesting metrics and insights about adoption, template usage and evolution over time, but it did not go into the actual ADR contents. For instance, given existing taxonomies or well-known types of design decisions reported in the literature [5, 6], the ADRs can address some of those decisions, cover other concerns, or even include non-architectural concerns. Moreover, it is important to assess whether the ADRs discuss quality-attribute aspects (e.g., performance, security, maintainability, or scalability, among others) in terms of drivers or consequences of the decisions. We argue that this content-based analysis can provide key indicators for understanding current AK practices and suggesting ways to improve them.

To address the gaps and perspectives above, in this work we conduct an empirical study of ADR contents using a dataset of open-source projects [4] and analyzing ADRs from different perspectives. Specifically, we examined ≈ 550 projects with a total of ≈ 4300 ADRs. Our objectives are to identify the main design concerns captured in ADRs, evaluate their alignment with existing decision types, and assess ADR sections (e.g., drivers, alternatives, rationale, consequences) for possible deficiencies. To this end, we formulate a number of research questions and define a data mining approach based on topic modeling techniques [7] and classification with the help of Large Language Models (LLMs) [8]. To calibrate the quality of the LLM output, we took a sample of ADRs from the dataset

EASE 2026, 9–12 June, 2026, Glasgow, Scotland, United Kingdom

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

[1] https://adr.github.io/madr/

[2] https://docs.aws.amazon.com/prescriptive-guidance/latest/architectural-decision-records/adr-process.html

and established a baseline through a manual analysis by experts. We also tested different prompt strategies (zero-shot, few-shots, chain-of-thought) for the classification task.

We report several key findings from our study, including the prevalence of existence decisions (from Kruchten's ontology [5]), the high proportion of decisions addressing quality attributes such as maintainability, security and reliability, the inclusion of technology and organizational concerns in ADRs, and the limited documentation of design alternatives when using the MADR template, among others. By revealing topics, gaps and inconsistencies in ADR contents, this study not only characterizes how architectural decisions are documented but also suggest improvement opportunities.

2 BACKGROUND AND RELATED WORK

Architecture Decision Records

An architectural decision can be defined as a justified design choice addressing a functional or quality-attribute requirement that is architecturally significant for a system. Such significance typically arises from the decision's long-term consequences or the high cost of modifying it later in the development process. In this context, an Architecture Decision Record (ADR) is intended to document a single design decision along with its rationale. Taken together, ADRs form a decision log that allows practitioners to capture, track, and communicate key decisions throughout the system's evolution.

The ADR concept was popularized by M. Nygard and others [2] as a lightweight, text-based template for decisions in software projects. The format emphasizes simplicity and ease of use, encouraging developers to treat ADRs as artifacts stored alongside the source code. Nygard's proposal includes fields such as: status, context, decision, and consequences, to provide a concise yet comprehensive snapshot of the rationale behind design choices. For instance, Figure 1 presents a typical ADR for a *GraphQL* data layer.

Since its introduction, ADRs have been widely adopted [1, 9]. Researchers and practitioners have proposed templates to accommodate different documentation needs, addressing challenges such as traceability, consistency, and discoverability of architectural decisions, especially in collaborative and long-lived projects. A prominent evolution of the ADR concept is the *Markdown Architectural Decision Record (MADR)* template, which is exemplified in Figure 2[3]. MADR builds upon Nygard's proposal but also adds elements (sections) from other decision documentation practices, such as Zimmermann's *Y-statements*[4]. MADR introduces additional fields to enhance clarity and traceability, such as: status (indicating the decision lifecycle, e.g., proposed, accepted, deprecated), decision drivers (key concerns or goals influencing the decision), considered options (alternatives evaluated during the decision process), and pros and cons (comparative evaluation of each option).

Previous Works on ADR Usage

The practice of capturing design decisions (with or without ADRs) is often not widespread in software projects due to several challenges. Falessi et al. [10] identified barriers such as time restrictions, a lack of process integration, and insufficient tool support, as common reasons for a poor documentation of decision reasoning. Buchgeher et al. [4] conducted a study based on mining information about ADRs from hundreds of Github projects. Their analysis revealed certain trends in ADR adoption, template usage, and evolution over time, demonstrating that ADRs are increasingly popular in the open-source community but often used in inconsistent or ad-hoc ways. Their analysis showed that overall adoption remains relatively low. For instance, in over half of the repositories containing ADRs, there were only one to five of them present, suggesting that many projects merely experimented with ADRs without fully integrating them as an ongoing practice. On the other hand, a small fraction of repositories (those with more than 20 ADRs, long-term maintenance, and contributions from multiple developers) exhibited signs of systematic ADR adoption, typically as part of collaborative team processes. The study highlighted barriers to broader adoption, including the difficulty of discovering ADRs across repositories due to their inconsistent naming and file structures. While ADR templates appear sufficient for many users, the authors stressed that many questions about ADR usage remain unanswered.

Ahmeti et al. [3] found that organizational inconsistencies in structuring ADRs can lead to low reusability and difficulty in maintaining them effectively across teams. Additionally, the authors reported that practitioners often struggled to determine whether a decision was architecturally significant enough to warrant documentation, highlighting a lack of guidance on what constitutes a "key" decision. The absence of clear criteria led to inconsistent documentation practices, with decisions being recorded based on subjective factors such as perceived complexity, impact on system implementation (e.g., technology, deployment), or number of stakeholders involved. Time constraints in agile development further exacerbated this issue, limiting the extent to which decisions could be thoroughly documented. The study also emphasized that the perceived usefulness of ADRs is closely tied to where and how they are stored, and that while ADRs can improve inter-team coordination, their effectiveness depends on well-defined processes and supporting documentation frameworks.

To mitigate the barriers above, there is an emergent trend towards applying LLMs to support the generation and refinement of ADRs. Dhar et al. [11] investigated whether LLMs could generate a design decision from a provided design context, and reported that LLMs were able to produce readable content but required additional structuring and validation. Building on this work, Dhar et al. [12] proposed a framework that combines retrieval-augmented generation with fine-tuned models to generate decision statements from short context paragraphs. However, the model was mainly asked to generate a decision that appears coherent and relevant to the input context, rather than asked to reason through trade-offs or provide justifications. While promising as a classification or summarization task, this line of work raises questions about its practical utility. Without surfacing rationale, alternatives, or quality concerns, the generated outputs may not reflect the kind of reflective process that ADRs are meant to support. This issue underscores the need for approaches that not just extract decisions, but help structure and analyze the architectural reasoning process itself.

Other works have focused on improving the quality of ADRs, especially when generated by novice architects. For example, Díaz-Pace et al. [13] proposed an LLM-based assistant, called *ArchMind*,

[3]Adapted from: https://ozimmer.ch/practices/2022/11/22/MADRTemplatePrimer.html
[4]https://medium.com/olzzio/y-statements-10eb07b5a177

ADR: GraphQL Data Layer

Status
Accepted, installation of Ariadne and Apollo complete. Creation of GraphQL schema in progress. (October 28, 2020)

Context
Boxwise is in the middle of a planned migration from the old PHP-based Dropapp, to a new app based on a new stack (Python/React). In the old app, the backend and the data model were closely intertwined, with SQL strings written directly within PHP. This made it challenging to evolve the data model for the app, which in turn imposed many product functionality constraints and slowed development time - especially since the Dropapp data model was a prototype rather than designed for scalability and product-market fit. As the team migrates to the new app and explores the possibility of entering new markets, it is time for us to reexamine if and how the team might benefit from a separation of concerns on the data layer.

Decision drivers
1. **Scalability**: how well will it support expected future changes such as DB restructuring, database migrations, etc.? What timescale is the technology expected to be defunct on?
2. **Developer experience**: given the rotating environment of loosely affiliated developers of different backgrounds, what will support the rapid onboarding of developers with our data structure? Once onboarded, is the chosen technology pleasant to use? Is it useful from a career progression standpoint?
3. **Maintainability**: we expect to have rapid changes to the DB structure as we expand functionality. How easy is our solution to maintain and evolve? What about documentation?
4. **Support and production-readiness**: Is the library mature enough to use in a production environment? Is there an active community or support channel should we run into problems?

Considered options
API Styles

Full REST interface with backend. Would involve the creation of multiple endpoints and resources that devs will have to request. Well-understood by professional devs, but new devs (coming from data analysis background, for example) would need to learn the correct REST standards. Has an over/underfetching problem, which causes it to be network traffic heavy. Difficult to evolve the API as you don't know what queries are requesting which fields from specific resources. Often leads to creating one endpoint per client.

No data layer / separation of concerns. Would require all devs to be fluent in current table structure. All changes in the data model will need to be paired with cascading SQL query changes (or the equivalent in an ORM).

Blended environment (GraphQL endpoint for inventory + REST endpoints for login). Logic here was that it might be easier to create a REST endpoint for login and users, as proof of concept had already started with REST, and overfetching / underfetching is less of an issue with users. Katie said she preferred to build everything in one style. Accepted this point as she will be implementing login.

GraphQL with single endpoint for everything. Benefits avoids underfetching/overfetching problem, very readable on the front-end, query language is super easy compared to learning SQL from scratch. Enables parameterized queries, and inherently supports incremental evolution as fields are explicitly specified in each query. Supported by Facebook and is being adopted by major tech companies such as Paypal, Github, eBay etc. Cons - new kid on the block, most devs are much less familiar with GraphQL as a concept. Requires devs to understand the concepts of queries, mutations, resolvers. N+1 problem. Less easy to cache than REST.

Server

Graphene: Takes a code first approach. Development is reportedly lagging behind with maintainers looking for people to take over. Oldest Python solution around, so likely quite stable. Many frustrated users on reddit.

Ariadne: Takes a schema-first approach. Excellent documentation, with functionality designed to mimic industry leader Apollo Server, which uses node.js and is not compatible with a Python back-end. Fewer stars compared to Graphene; however, compensates for this somewhat through their Spectrum support channel. Well-loved on reddit, but on 0.11 release. Supported by a small dev shop, Mirumee Software.25

Client

Apollo Client: Well-supported industry leader. Extensive documentation, sophisticated caching solution. Large footprint. Previously considered hard to set up and configure due to sophistication, but now has Apollo Boost package to make things super simple and speedy.

urql: A lightweight tiny client solution intended to make graphQL simple. However, it lacks one of the major benefits of the Apollo client, which is cacheing. Supported by a medium-size dev shop. Younger than Apollo Client as well as all of the server options.

Decision
GraphQL with single endpoint for everything was selected, paired with Ariadne server-side and Apollo client side.

Reasoning: while GraphQL may have a steeper learning curve for professional developers who are not familiar with the standard, in the long run this should be more scalable for iterations and easier to maintain than multiple REST endpoints. In the future, should we end up ingesting external data APIs such as the UNHCR data, it will be easier to pull that all from the GraphQL endpoint as well. This should also be more favorable from a developer experience standpoint for both onboarding and maintaining the codebase, due to GraphQL's introspective capabilities, human-readable JSON query structure, and degree of client-side specificity in requesting field-level data.

Apollo was selected on the client-side due to its maturity as a product, robust features including sophisticated caching, excellent documentation, and huge community. Ariadne was selected over Graphene on the server side due to it being designed to deliberately intended to mimic Apollo Server. With Ariadne being under active development by Mirumee software, and its excellent documentation that developers can cross-reference with Apollo server documentation, I believe this outweighs any cons that come from it being a less mature library than Graphene.

Finally, I believe any performance concerns that could result from queries being abstracted from SQL into resolvers will be compensated for by less load on the network due to overfetching, as long as no N+1 queries are created.

Consequences
Easier

requesting data from the backend, once initial set up is complete
integrating external data sources
readability of queries
making changes to the database without breaking every single existing query
data structures (understanding and keeping track of the data structure and their relationships with one another)
versioning

More difficult

Initial set up means that we cannot take advantage of Flask utilities to create REST endpoints for things like login and routing
Initial set up of data schema
Optimizing for query performance
Error handling (GraphQL does not inherently use HTTP response codes like REST) potentially optimizing scalability and performance within a large scale distributed system

Further Reading

https://goodapi.co/blog/rest-vs-graphql

Figure 1: Full-fledged ADR example (taken from the https://github.com/boxwise/boxtribute repository)

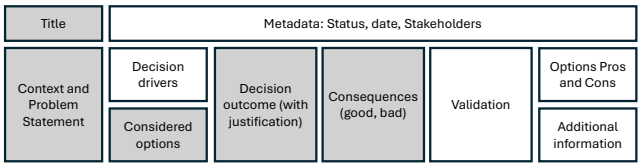

Figure 2: Representation of a typical ADR template

which is designed to guide novice practitioners in reflecting on architectural trade-offs. *ArchMind* combines architectural knowledge (e.g., patterns) with system-specific information to assist users in selecting and evaluating alternatives. In a classroom setting, the authors reported that this assistant helped students produce more concrete and better-justified ADRs, even if some context-specific elements were occasionally missing. Along with the work by Dhar et al. [12], this work highlights the potential of LLM-based tools for automating documentation and fostering architectural practices.

While the studies so far have advanced the understanding of how ADRs are used and how they can be generated, an important gap remains in the literature: *what do ADRs talk about?* We argue that existing work has paid scarce attention to the substantive content captured in ADRs. This paper takes a step towards addressing this gap by analyzing the thematic and structural content of ADRs at scale, considering the types of decisions, concerns, and quality attributes covered in practice.

Types of Architecture Decisions

To better structure and understand architectural decision-making, several taxonomies have been proposed in the AK literature. These taxonomies categorize the types of decisions made, the knowledge they capture, and their relationship to quality attributes or architectural intent. One of the first and widely referenced models is Kruchten's ontology of architectural decisions [5], which identifies three types of decisions commonly made during architecture design:

Existence (ontocrisis)	These decisions declare that an element or artifact will exist in the design or implementation. Includes structural decisions (e.g., layers, components) and behavioral decisions (e.g., connectors, interactions). Structural decisions lead to the creation of subsystems, layers, partitions, or components. Behavioral decisions are more related to how the elements interact together to provide functionality or to satisfy some non-functional requirement (quality attribute), or connectors.
Ban/Non-existence (anticrisis)	These decisions declare that an element will not exist in the design or implementation and often used to rule out alternatives.
Property (diacrisis)	These decisions state a general, enduring quality or constraint of the system. These are often cross-cutting concerns or design rules (positive) or constraints (negative).
Executive (pericrisis)	These decisions refer to decisions that do not relate directly to the design elements or their qualities but are driven more by the business environment (financial), and affect the development process (methodological), the people (education and training), the organization, and to a large extend the choices of technologies and tools.

Table 1: Kruchten [5]'s architectural decision ontology

existence decisions, *property decisions*, and *executive decisions*. These decisions can include attributes such as rationale, scope, status, cost, and risk, among others. Furthermore, decisions can have relations with each other to denote constraints, enablement, conflicts, or subsumption relations, among others. For the purpose of this work, we focus on the three main classes, which are summarized in Table 1. Our ADR in Figure 1 can be categorized as an existence decision.

An expert survey [14] found that ≈ 65% of the architectural decisions made by practitioners are existence decisions, primarily structural ones, while executive and property decisions are less common, comprising 27% and 8% respectively. Notably, non-existence decisions were not identified, despite their importance. As Kruchten [5] emphasized, such decisions are critical to document because they are not evident in the system structure or implementation and can easily be overlooked.

On the other hand, a design decision can be classified according to its *main concern or focus* [9], for example, choosing a programming language, architectural pattern, deployment technology, or middleware asset, could all have architectral implications. In this regard, Zimmermann et al. [15] and Zimmermann [16] introduced a classification as part of their reusable architectural decision model, including: design, technology, infrastructure, organizational/process, constraints, quality attributes, crosscutting concerns, or implementation decisions, as summarized in Table 2. Our example ADR is a *design* decision but also includes *technology* concerns.

The consideration of quality attributes is foundational to architectural reasoning and knowledge management [1], as decisions can target functional requirements or quality properties, and also have side-effects on them. The latter two aspects often entails architectural significance [17]. We adopt a classification of quality attributes grounded in the *ISO/IEC 25010 Standard for System and Software Quality Models*[5], complemented with the attributes proposed by Bass et al. [17]. In particular, we considered performance, reliability, security, maintainability, scalability, usability, portability,

[5]https://www.iso25000.com/index.php/en/iso-25000-standards/iso-25010

Design	They concern the logical organization, structure, and decomposition of the system. Related to patterns, components, layering, interfaces, data modeling.
Technology	They concern the selection of technologies, platforms, frameworks, libraries, or standards.
Infrastructure	They involve the deployment environment, hosting, runtime platforms, networking, and hardware concerns.
Organizational Process	They concern team structures, roles, responsibilities, workflows, and processes that affect architecture.
Constraints	They refer to mandatory conditions from the business, regulations, or existing systems that limit architectural choices.
Quality-Attribute	They explicitly target system qualities like performance, security, availability, etc. These are decisions made primarily to meet a non-functional requirement.
Crosscutting Concerns	They relate to decisions that impact multiple parts of the system simultaneously, often aspects like logging, monitoring, security mechanisms.
Implementation	They affect internal code structure, patterns at the class or method level, or maintainability mechanisms, but are not architectural in scope.

Table 2: Zimmermman classification of decisions based on their main concern

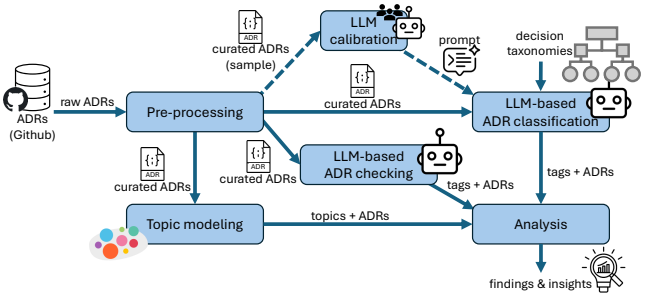

Figure 3: ADR mining and processing pipeline.

compatibility, and testability. Coming back to our example ADR, it mentions *performance*, *scalability* and *maintainability*.

3 STUDY DESIGN

The large number of ADRs in our dataset makes manual examination and annotation infeasible. At the same time, relying solely on statistical text-mining techniques is often not sufficient to capture the architectural intent and decision semantics embedded in ADRs. To balance scalability and interpretability, we therefore designed an automated analysis pipeline that combines topic modeling with LLM–based classification and checking. This hybrid approach enable us to uncover recurring themes in ADRs while mapping their contents to established architectural decision taxonomies and template structures. The pipeline consists of the following six components, as outlined in Figure 3:

1) *Pre-processing*. It involves the collection of ADRs from projects in open-source repositories, an explorative data analysis of the ADRs (documents), and the application of exclusion and inclusion criteria for our study. The outcome of this stage is a curated dataset.

2) *Topic modeling.* It performs an unsupervised analysis of the dataset to identify latent topics in the documents. We apply embeddings on the textual contents of the ADR documents and group them using clustering techniques.

3) *LLM prompting and calibration.* It generates a small ground-truth dataset used to assess classification quality and calibrate the LLM-based labeling process.

4) *Classification based on decision taxonomies.* It maps the contents of every document to predefined categories of decisions according to a taxonomy. We consider three existing architectural taxonomies and apply text classification using an LLM.

5) *Checks using MADR template.* It scans the documents and identifies correspondences with the MADR sections, determining whether each section is present and fulfills its intended purpose. This evaluation goes beyond standard text classification and leverages an LLM-based approach.

6) *Analysis and insights.* Based on the outputs of the previous components, both quantitative and qualitative analyses are performed. These analyses were driven by our research questions.

We tackle the following research questions:

• **RQ#1.** *What types of concerns (as topics) are captured in ADRs?* Here, we explore concerns in a broad sense, without ties to existing taxonomies or architectural concepts.

• **RQ#2.** *How well does the captured content align with the taxonomies from the literature?* Here, we analyze the proportion of ADRs that reflect Kruchten's decision types, examine their primary concerns using Zimmermann's framework, and identify which quality attributes appear most or least frequently.

• **RQ#3.** *Are there mismatches or inconsistent practices in the usage of the ADR structure (i.e., MADR sections)?* Here, we assess whether section titles (e.g., decision drivers, consequences, alternatives) accurately reflect their corresponding content, and also identify cases where these sections are omitted or misused.

In our study, we followed the guidelines for mining software repositories from Vidoni [18], which comprise: definition of sources, search for repositories, data extraction, and information generation.

All LLM-related experiments were conducted with `GPT-4.1-mini` model. The LLM was accessed via the `LangChain` framework[6] with a temperature set to 0 to minimize randomness in responses[7]. Each ADR was processed independently without conversational memory.

3.1 Data Collection

Our dataset of ADRs from open-source GitHub repositories was originally compiled by Buchgeher et al. [4] and covers projects in the 2013-2020 period. The dataset, provided in JSON format, includes metadata such as repository URLs and ADR file locations. Since our analysis does not focus on temporal evolution, we retrieved the latest available version of each ADR. If there were no ADRs in the most recent commit of a repository, the last commit referenced in the original dataset was retrieved. In some cases, repository names had changed and required manual correction.

We performed a cleaning step to ensure data quality. ADRs were discarded if they were duplicated, not written in English, or if they were only templates for creating real ADRs. Additionally, we filtered out projects containing fewer than five ADRs or whose ADRs were shorter than 500 characters on average, under the assumption that they would provide insufficient content for our analysis. From the original 867 projects we retrieved 6061 ADRs. After filtering, our final dataset consists of 4316 ADRs from 547 (unique) projects.

3.2 Preprocessing

To extract relevant information from the ADRs, we developed a custom Markdown parser. Despite Markdown's standardized syntax, the variability in how organizations structure their ADRs posed parsing challenges. While some ADRs followed well-defined templates like MADR, others deviated considerably. We adapted the parser to identify such cases to avoid missing key content. Heading levels were also applied inconsistently, which complicated structural interpretation of sections. Extracting the main "Decision" section proved particularly challenging, as it could appear at various levels of the hierarchy, be embedded within broader sections, or be accompanied by subsections like "*Pros*", "*Cons*". Certain sections were empty or included placeholders (e.g., "*TBD*" or "*Add decision*").

3.3 Topic Modeling

`BERTopic` [19][8] is a topic modeling approach that leverages transformers, clustering and TF-IDF techniques to generate coherent topic representations of documents. In our context, a document is equivalent to an ADR; thus, we can extract the main topics covered by the ADRs. In particular, `BERTopic` first creates document embeddings to obtain document-level information, and then reduces the dimensionality of the embeddings before creating semantically similar clusters, each one representing a distinct topic. A class-based variation of TF-IDF is used to extract the topic representation from each cluster. Basically, all documents within a cluster are treated as a single document or class (by concatenating their contents). A TF-IDF strategy is adjusted to account for this representation. This procedure, called *c-TF-IDF*, permits to capture the importance of words in clusters rather than individual documents. The number of topics in the c-TF-IDF representation can be reduced by iteratively merging the least common topic with its most similar one.

Traditional topic modeling techniques such as Latent Dirichlet Allocation (LDA) were considered, but they rely on bag-of-words assumptions that often produce sparse and less coherent topics for short, domain-specific texts like ADRs. `BERTopic` was preferred because it leverages contextual embeddings from transformer models to capture semantic similarities across heterogeneous vocabulary, yielding more meaningful and fine-grained clusters for our artifacts. To assess topic coherence and architectural relevance, two authors manually inspected a sample of ADRs associated with each topic, together with their keyword representations. This qualitative validation was used to analyze cluster consistency and check for representative topic labels. While this step was not intended as a full qualitative coding exercise, it provided confidence that the identified topics were meaningful from an architectural perspective.

3.4 Prompting and Calibration

LLMs represent a paradigm shift in natural language processing, enabling tasks such as classification through instructions rather

[6] https://python.langchain.com

[7] https://platform.openai.com/docs/api-reference/chat

[8] https://maartengr.github.io/BERTopic/index.html

than task-specific training [20, 21]. This capability makes LLMs particularly suitable for our context, where little (or no) labeled data is available and ADR content varies across repositories and projects. Given the unstructured format of ADRs and the scale of our dataset, manual annotation of all ADRs would have been prohibitively time-consuming and impractical. For these reasons, we designed an LLM-based classification approach and designed prompt instructions for the three taxonomies described in Section 2, which were applied consistently across the entire corpus.

Ground-truth construction.. To support the evaluation of automated classification strategies, we constructed a manually annotated subset of ADRs. We selected a random sample of 200 ADRs from different projects and manually annotated them according to the three considered taxonomies (Kruchten's, Zimmermann's and quality attributes). Two of the authors acted as reviewers and independently labeled each ADR. To assess annotation consistency, 50% of the sampled ADRs were evaluated by both authors (i.e., overlapping items). Disagreements and potential misclassifications were subsequently discussed until consensus was reached. This annotated subset constitutes a (small) *ground-truth* dataset for the study, although not necessarily covering all the ADR cases.

Embedding-based baseline.. Using the constructed ground-dataset, we first investigated whether a traditional supervised classification approach would be sufficient for the task. For each taxonomy, we trained a multi-class classifier based on gradient boosting (*XGBoost*) using document embeddings (*allMiniLM-L6-v2*) extracted from ADR texts. However, this approach yielded limited performance. The classifiers exhibited strong sensitivity to class imbalance, and consistently favored majority classes (e.g., existence decisions in Kruchten's taxonomy; maintainability and security for quality attributes; and design and technology in Zimmermann's framework). Moreover, confusion-matrix analysis revealed poor separability among classes, indicating that embeddings alone are insufficient to capture the semantic intent and pragmatic function required for taxonomy-based ADR classification. These limitations motivated the adoption of an LLM-based classification approach capable of reasoning over architectural intent, modality, and contextual cues beyond surface-level semantic similarity.

LLM prompt refinement.. For the ADRs included in the ground-truth dataset, we compared the labels assigned by human experts with those inferred by the LLM. Throughout the study, the LLM was instructed to act as *a software architecture expert tasked with analyzing Architectural Decision Records and classifying them according to a given taxonomy of decision types.* All taxonomies shared a common prompt structure, into which taxonomy-specific definitions, rules, and examples were injected. Building on this, we evaluated three prompting strategies[9], following an iterative prompt-refinement process[10] aimed at improving classification reliability.

The first strategy corresponded to a *zero-shot* configuration, in which the LLM was asked to assign a primary category to an input ADR based solely on the taxonomy description and classification instructions, without examples or additional architectural guidelines. Although the task was formulated primarily as single-label classification, the prompt allowed the model to return alternative categories together with qualitative confidence levels in a structured JSON response. An analysis of discrepancies between human annotations and zero-shot LLM outputs revealed several weaknesses in the initial prompt formulation. Common sources of confusion included the distinction between executive and existence decisions in Kruchten's taxonomy; the dominance of maintainability over related quality attributes such as portability or compatibility; and subtle boundaries between design, technology, and implementation concerns in Zimmermann's framework. Organizational and process-related decisions were also occasionally misclassified as product-level concerns, further motivating the need for more explicit prompt guidance.

For the second strategy, as the zero-shot strategy exhibited limited agreement with the ground truth, we extended the prompts with *static few-shot* examples, providing one or more representative ADR excerpts per category. In addition, we introduced chain-of-thought-style reasoning instructions and explicit disambiguation rules for frequently confused categories. As a third strategy, we employed *dynamic few-shot* prompting, in which a small number of examples were automatically selected from the ground-truth dataset based on semantic similarity to the input ADR and injected into the prompt. For this strategy, we also leveraged the chain-of-though prompting mentioned above.

3.5 LLM-based ADR classification

Each ADR, after preprocessing, was provided to the LLM in its entirety, with only consecutive blank lines removed in order to preserve structural cues. The model was instructed to return structured outputs in the form of a JSON object containing: (i) the primary selected category, (ii) a brief explanation justifying the classification, and (iii) a confidence score (in the $0-1$ range) for each alternative category. To ensure consistency and limit hallucinations, the LLM was explicitly instructed to rely exclusively on information explicitly stated in the ADR and to avoid inferring unstated intent.

Using the prompting configurations defined in the previous component, we applied the LLM-based classifier to the ADRs in the ground-truth dataset. Across the evaluated taxonomies, we observed agreement rates ranging from 46% to 66% for Kruchten's taxonomy (4 classes), Zimmermann's taxonomy (8 classes), and the quality-attribute classification (10 classes), across the evaluated prompts. As expected, classification quality varied across taxonomies and was influenced by the adopted prompting strategy, with higher agreement generally achieved when moving from zero-shot to few-shot configurations.

Overall, fewer discrepancies were observed for Kruchten's taxonomy, more for Zimmermann's categories, and intermediate behavior for quality attributes. These differences can be attributed both to the stochastic nature of LLMs and to the inherent subjectivity of certain taxonomies. For example, an ADR may address multiple quality attributes simultaneously or combine design decisions with technology choices and implementation details, which complicates classification within Zimmermann's framework.

[9]Examples of prompt evolution are available in the companion repository: https://anonymous.4open.science/r/ADRminer-716D/notebooks/prompts.py

[10]https://llm-guidelines.org

Based on the comparative evaluation of the prompting strategies, we selected the best-performing configuration for each taxonomy. In particular, the refined static few-shot strategy proved especially effective for quality-attribute classification, while the dynamic few-shot approach yielded the best performance for Kruchten's and Zimmermann's taxonomies, albeit at the cost of increased latency and token consumption. Finally, the complete ADR corpus was classified using the selected prompting strategy for each taxonomy.

3.6 LLM-based ADR Checking

To analyze the structural consistency of ADRs, we focused on the key sections of the MADR template: context, decision, consequences, decision drivers, and considered options. For each section, the following criteria were defined:

• *Presence.* Whether a given section (i.e., its heading) is explicitly present in the ADR.

• *Alternative title.* When the expected heading was missing, we evaluated whether the content for that section appeared under different headings, recording those headings (if found).

• *Content quality.* For each section (or its detected alternatives), we assessed whether the text had meaningful, project-specific content rather than vague or generic statements.

• *Purpose consistency.* We examined whether the content fulfilled the intended purpose of the section, as well as with the roles of other sections. We used a three-level scale for this criterion. A *Yes* value indicated that the content was clear and well-scoped, fulfilling the intended section role. A *Partial* value was assigned when some degree of overlap with another section was observed. Finally, a *No* value denoted that the content either belonged to another section or failed to fulfill the intended purpose of the section.

The four criteria were implemented using an *LLM-as-a-judge* strategy [22]. For ADRs in which the target section heading was present, the LLM was asked to evaluate content quality and purpose consistency, and provide a justification for the responses. When the heading was absent, the LLM was additionally prompted to search for suitable content within the ADR text. Furthermore, the LLM received a description of the section's intended purpose. To avoid confusion and ensure focused responses, each ADR section was independently analyzed by prompting the LLM separately[11]. Throughout the analysis, the LLM was asked to strictly adhere to our evaluation guidelines, namely: use all available information from the ADR text, assume minimal external context, favor clarity over guesswork, and be conservative in section assessments. Sections that were duplicated or overlapped with others were penalized accordingly, and placeholders were marked as misuse unless appropriately replaced with project-specific information. To assess the reliability of the LLM-as-a-judge evaluations, two of the authors independently reviewed a random sample of ADR assessments and confirmed the consistency of the resulting judgments.

4 ANALYSIS RESULTS

4.1 RQ#1 - Types of Concerns as Topics

The number of topics was not pre-specified but resulted from `BERTopic`'s iterative topic reduction step, which merges low-frequency clusters with their most semantically similar neighbors based on cosine similarity between embeddings. The resulting 67 topics provided a good balance between coherence (0.38) and diversity (0.96)[12]. These results indicate moderate internal consistency among topic terms and a very high differentiation across clusters, ensuring that topics were semantically distinct without fragmenting related ADRs into overly fine-grained clusters.

Figure 4 shows the main topics identified by `BERTopic` in the ADR dataset. Each point in the chart[13] corresponds to an ADR and the 2D spatial distribution tries to preserve the semantic distance among ADRs via a dimensionality reduction technique. We used *all-MiniLM-L6-v2* as the sentence transformer to generate document embeddings in `BERTopic`. The topics are visualized as clusters of related ADRs. Although a document (i.e., an ADR) can be affected by several topics, only the most prevalent topic is shown. Those ADRs with unclear predominant topics are marked as noise (gray points), and they often lie in the cluster boundaries.

Based on the keywords (topic representation) returned by `BERTopic` for each topic, we assigned a representative label to the topic. The key topics were related to *Authentication and Authorization* (security), *Deployment* (Kubernetes), *Branches and Releases* (versioning), *API Development* (OpenAPI), *NoSQL databases* (Cassandra, cloud providers), *blockchain* (Tendermint), and *frontend* (CSS). A detailed description of the topic representations along with their labels is given in Table 3. We grouped the topics into categories by leveraging a hierarchical topic organization feature provided by `BERTopic`. From Figure 4, we can notice a mixture between development-related concerns and specific technologies, including certain references to the concerns of Table 2 and very few quality-attribute concerns (security, interoperability). An in-depth inspection of the topics in Table 3 reveals topics related to *databases*, *security*, *blockchain protocols*, *cloud infrastructure*, and *software development tasks* (APIs, branches, telemetry, wikis, programming languages). The latter category is kind of unexpected from an AK perspective, but the dataset shows that it seems to be a common practice among practitioners. Also, *infrastructure* and *deployment* aspects were identified as recurring topics in ADRs, which are accompanied by technology mentions (e.g., AWS, Python, Java, React, RabbitMQ, Redit, Kubernetes, or Maven, among others).

While Table 3 lists topics that primarily address technology and operations (e.g., authentication, deployment, API development), many of these directly relate to established AK categories. For instance, topics on deployment and Kubernetes map to *infrastructure* concerns, authentication and cryptography relate to *security* (as a quality attribute), and frontend and styling decisions fall under *design and usability*. This mapping reinforces that ADRs capture

[11] Preliminary experiments revealed that simultaneous analysis of multiple sections can lead to inconsistent and erroneous outputs.

[12] Perplexity metrics, typically used for probabilistic models like LDA, do not apply to `BERTopic` since it is embedding-based rather than generative. Instead, coherence and diversity jointly capture the internal quality and external separability of the topics.

[13] An interactive version of Figure 4 for exploring topics and ADRs is provided in the companion repository.

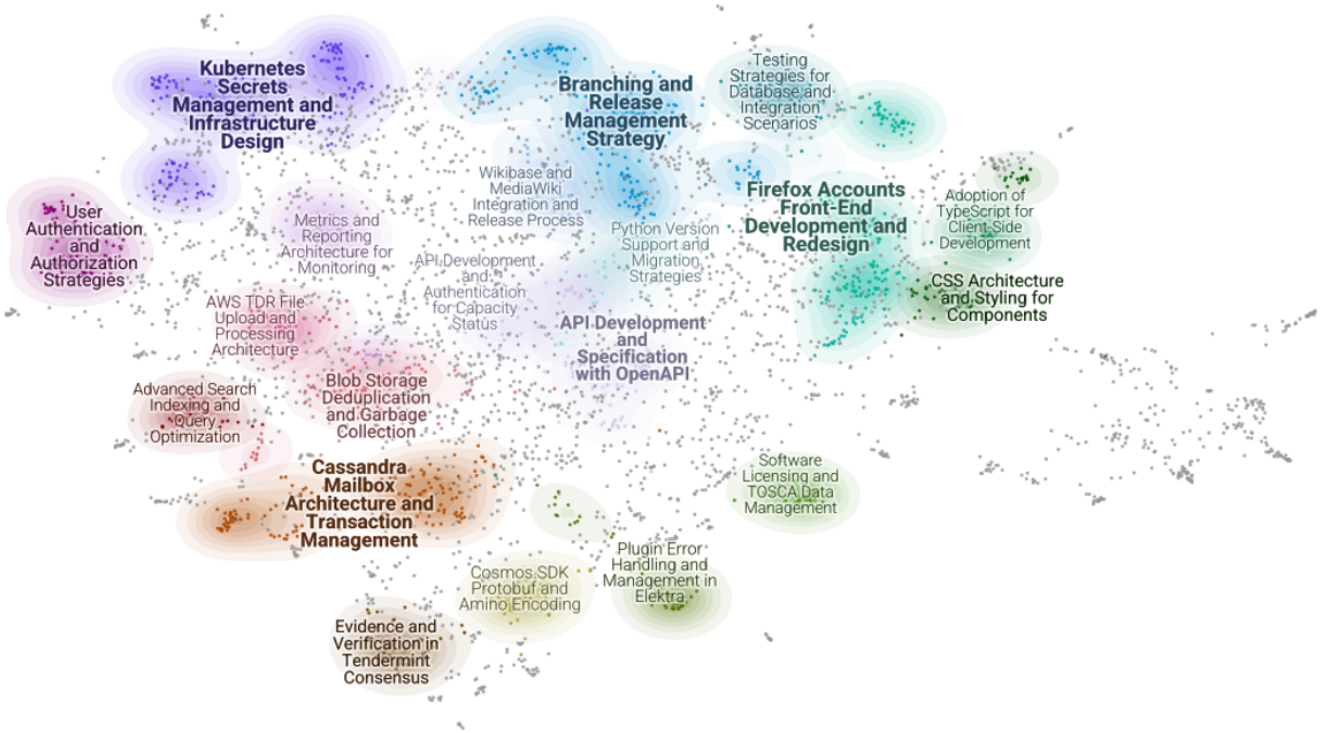

Figure 4: Top-20 topics detected in the ADR contents (topic font size is proportional to the topic prevalence in the corpus)

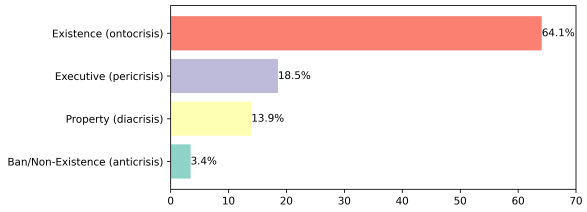

Figure 5: ADRs according to Kruchten's decision taxonomy

not only architectural structures but also supporting technologies and crosscutting concerns.

4.2 RQ#2 – Relations with Decision Taxonomies

We analyzed ADRs according to the three presented taxonomies. The results of classifying the ADRs according to Kruchten's taxonomy are summarized in Figure 5. If an ADR shared multiple decision types, the type with the highest confidence (as predicted by the LLM) was considered. The chart shows a significant proportion of *existence* decisions (64%), followed by a smaller proportion of *executive* decisions (18%), and very few *property* (14%) and *non-existence* decisions. Although these numbers are aligned with the trends reported by [14] (for a sample of 120 ADRs), we found a similar number of existence decisions (65%), fewer executive (27%) decisions and more property decisions (8%).

The emphasis on existence and property decisions makes sense for ADRs, as they are expected to be assertive about design structures, behaviors, constraints and crosscutting concerns. Existence decisions are related with *infrastructure* (e.g., Terraform, Kubernetes, AWS, Docker) but also with testing, versioning, repository structure, authentication, API management, OSS, and event-driven components. Property decisions involved error handling and identifier usage. The low proportion of executive decisions, which often refer to business or methodological considerations, can be due to the technical nature of ADRs. Executive decisions were linked to CI/CD considerations and documentation practices. A few ADRs in this category had technology considerations (AWS). Nonetheless, most technology aspects mentioned in ADRs were primarily tagged as existence decisions.

Interestingly, our approach identified a very small fraction of *non-existence* decisions (43 ADRs). These decisions explicitly stated not to use certain technologies (e.g., PaaS feature, React component, Hawk authentication, Docker), or design mechanisms such as transactions (in smart contracts), web scrapping, dynamic memory, local data, or content types. In some cases, the ADRs referred to deprecation scenarios or warned about dependencies on external packages that should be avoided. This finding shows evidence that architects do rely on non-existence decisions when necessary.

Despite our prompt improvement efforts, the LLM classifier sometimes confused executive and existence decisions, which can be due to the lack of enough context (in the ADR itself) about what constitutes the product/system and the surrounding business/development environment for it. For a better classification, the LLM prompt should incorporate the project context, which is usually implicit in the ADR text.

Regarding quality-attribute concerns, Figure 6 depicts the coverage of the different attributes across ADRs. The most prevalent attributes were: *Maintainability*, *Security*, *Reliability*, *Performance*, in addition to *Functional* aspects. *Usability* and *Compatibility* were present to a lesser extent. We observed a gray area to distinguish

Category	Topic	Description	Representation	Size
Data Management and Security Strategies	3	Cassandra Mailbox Architecture and Transaction Management	cassandra, rabbitmq, queue, transactions, mailbox, imapuidtable, messageidtable, atlasdb, transaction	142
	5	User Authentication and Authorization Strategies	authentication, authenticate, auth, oauth, auth0, authorization, oauth2, authenticated, credentials, sessiontoken	88
	6	Blob Storage Deduplication and Garbage Collection	blobstore, storage, blobs, storing, blob, stored, preservation, deletions, store, files	82
	11	Plugin Error Handling and Management in Elektra	plugins, specification, variants, exceptions, elektra, apis, warnings, plugin, errors, encapsulating	53
	13	AWS TDR File Upload and Processing Architecture	aws, s3, uploading, uploads, cloudfront, cloudwatch, cloud, upload, buckets, bucket	45
	20	AWS RDS Database Architecture and Storage Decisions	nosql, rdms, aws, rds, databases, postgres, storage, db, postgresql, database	33
Cross-Platform Architecture and Interoperability	14	Python Version Support and Migration Strategies	pip, python, py, packages, repository_tool, versions, libraries, tuf, implementations, dict	45
Frontend Development and Code Quality Practices	7	CSS Architecture and Styling for Components	styles, stylesheets, styling, stylesheet, css, styled, sass, typescript, components, style	77
Interoperability and Security in Blockchain Protocols	8	Evidence and Verification in Tendermint Consensus	evidence, attacks, validators, consensus, protocol, amnesia, validator, witness, conflicting, attack	67
	9	Cosmos SDK Protobuf and Amino Encoding	cosmos, sdk, protobuf, compatibility, serialization, signdoc, protocol, proto, encoded, pubkeys	61
	60	Cryptographic Protocols and Key Exchange Methods	protocol, secrecy, otr, otrv4, otrv3, encrypted, messages, conversations, security, prekey	13
Open Source Licensing and Software Management	18	Software Licensing and TOSCA Data Management	eclipse, xml, epl, java, licenses, xsd, license, apache, repository, copyright	36
Optimized Search and Indexing for Genomic Data	15	Advanced Search Indexing and Query Optimization	elasticsearch, lucene, searchservices, indexing, searching, indexes, indexed, search, solr	42
Software Development and Integration Strategies	0	Firefox Accounts Front-End Development and Redesign	react, jsx, fxa, framework, webpack, redux, frontend, typescript, backbone, components	170
	1	Branching and Release Management Strategy	branch, commit, release, git, workflow, github, deployment	170
	2	Kubernetes Secrets Management and Infrastructure Design	kubernetes, terraform, cloudformation, pods, pod, aws, cloud, secrets, k8s, elasticsearch	162
	4	API Development and Specification with OpenAPI	swagger, api, openapi, documentation, specification, yaml, restful, schemas, versioning	108
	10	Testing Strategies for Database and Integration Scenarios	testcontainers, testing, testharness, mock, mocking, test, coverage, testdatagen	60
	12	Metrics and Reporting Architecture for Monitoring	monitoring, prometheus, analytics, metrics, dashboards, cloudwatch, dashboard, instrumentation, influxdb, alerts	49
	16	API Development and Authentication for Capacity Status	api, services, status, service, requests, backend, django, capacity, servers	40
	17	Wikibase and MediaWiki Integration and Release Process	wikibaseclient, wikibaselexeme, wikibaserepo, wikibase, mediawiki, wikidata, wiki, release, documentation	37

Table 3: Summary of the top-20 topics extracted from the ADRs. For better readability, topic representations were linked to descriptions, and descriptions were grouped into categories.

decisions with a strong functional component and very limited quality emphasis from decisions where quality attributes were more prevalent. Maintainability also tended to dominate (or absorb) other related qualities such as compatibility or portability.

When crosslinking Kruchten's results with the quality-attribute decisions, we found that most existence decisions had to do with maintainability, security reliability, and functionality, while executive decisions involved maintainability, security and functionality. Maintainability topics alluded to logging, testing, code quality, and systems using React, TypeScript and CSS. The usage of testing tools seemed to be an important concern. Compatibility was related to technology integration, while performance was related to caching, queues, and P2P networks, often paired with reliability or scalability. As for Security, it was associated to secure communications, encryption, authentication and authorization. The fact that several ADRs were tagged as primarily functional is an indicator of a practitioners' need to capture those concerns in the code repository.

The results for Zimmermann's classification are shown in Figure 7. As expected, almost 40% of the ADRs were classified as *Design* decisions, and a quarter of them were related to *Technology*. The difficult cases for the LLM had to do with (subtle) distinctions between design, technology, or implementation, and with decisions related to organizational issues or processes that were mistakenly associated with the product or system being built (and tagged as design or technology, thereof).

With respect to Kruchten's taxonomy, we corroborated that most existence decisions referred to design or technology concerns. On the other hand, this analysis provided a new perspective regarding *Organizational/Process* concerns, as ≈ 20% of the ADRs belonged to that category. These decisions talked about change logs, CI/CD

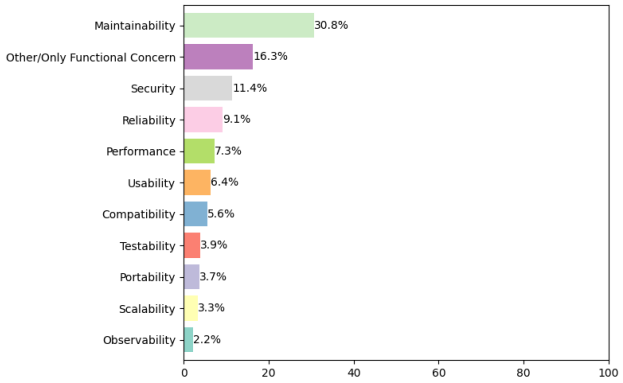

Figure 6: Prevalence of quality-attribute concerns in ADRs.

pipelines, configuration management, security policies, metadata management, approval workflows, testing, and feature flags, with a clear trend towards the software development lifecycle. More than half of the process decisions were existence decisions, while the rest was divided among executive and property decisions. The decisions about *Crosscutting Concerns* were mostly existence decisions, but some of them were property decisions (e.g., observability, access management, coding style, error handling).

4.3 RQ#3 – Usage of ADR Template Sections

To assess the structural compliance of ADRs with the MADR template, we employed an LLM-as-a-judge approach to determine whether each section was present and whether its content fulfilled the intended purpose. For each evaluation criterion, we computed the percentage of ADRs that complied with the condition under

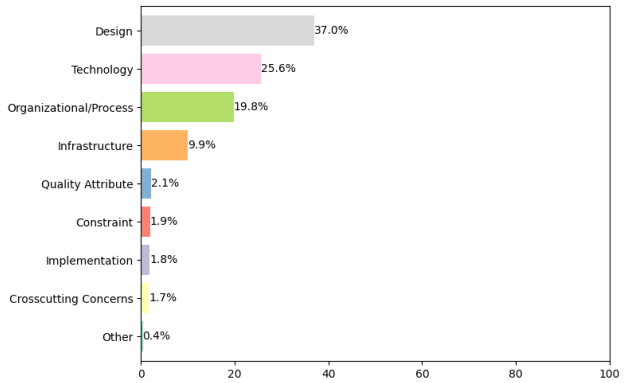

Figure 7: ADRs according to different decision types.

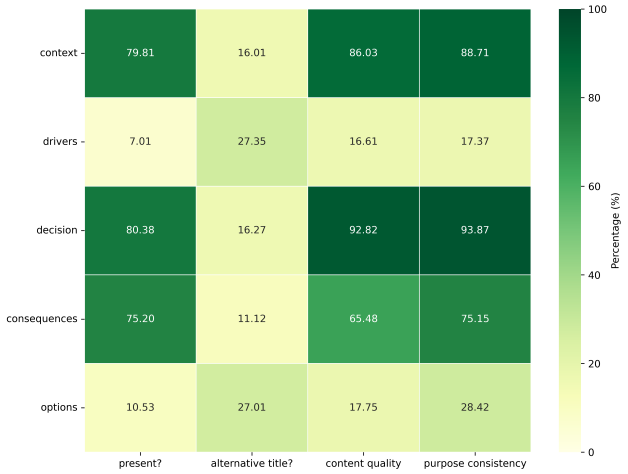

Figure 8: Degree of compliance of ADR sections to evaluation criteria (e.g., section presence, content quality, etc.).

analysis. The results are reported in Figure 8, where higher percentages indicate stronger adherence to the MADR template. When the purpose-consistency criterion was assessed as partial by the LLM, we conservatively treated it as a satisfactory case in the analysis.

The *Context* and *Decision* sections were mostly present, under the right template section, and including relevant contents. The *Consequences* section (for the main decision) was placed in the correct template section but with less relevant content. For the *Decision Drivers* section, we found that it was usually misplaced in the template (e.g., as part of the context or as a preamble for the main decision) and also had poor quality. The most surprising result was the case of the *Considered Options* section, because it was absent or sometimes misplaced in most ADRs, and when present, the section content did not meet its purpose.

The low scores for *Decision Drivers* and *Considered Options* can be seen as ADR "smells" that signal deficient design practices. There appears to be a direct correlation between the lack of good *Decision Drivers* and the omission of *Considered Options*. In particular, the issues with the *Considered Options* section suggest that architects do not capture alternative decisions, or probably they even disregard the analysis of alternatives to the main decision. Although this finding needs to be further investigated, the (mis)practice can undermine the rationale of the architecting process and contribute to AK vaporization in projects.

5 DISCUSSION

Our findings provide insights on how ADRs are actually documented in terms of common topics, and organized around the MADR template, which uncovers opportunities for enhancing the ADR authoring process. The process of building LLM-based classifiers also sheds light on ambiguity cases when categorizing ADRs. Furthermore, our content-based evaluation highlights which ADR sections and decision types are most often under-documented.

For practitioners, results show that ADRs from open-source projects are useful but should not be taken as "good examples" of standard documentation practices for design decisions, given the observations discussed in Section 4. Probably, practitioners should perform more rigorous reviews of the ADR sections and contents (e.g., when making a pull request cycle). Our analysis also challenges the assumption that ADRs exclusively capture architectural concerns (e.g., patterns, quality attributes). In practice, they often include less-architectural yet relevant aspects such as deployment, code, and process-related decisions.

The frequent omission of sections such as "Decision Drivers" or "Considered Options" in the MADR template could plausibly stem from practical constraints in agile development environments, where time pressure may hinder comprehensive documentation. Practitioners might also perceive these sections as redundant when rationale is already discussed in code reviews or issue trackers. Although these explanations remain hypothetical, they provide directions for future user studies with practitioners.

When it comes to researchers, we see a need for longitudinal studies [4] to better understand how ADR practices evolve over time, across project phases, and in relation to tooling and team composition. Regarding tool support, the usage of ADR templates could be improved with (automated) tags and section suggestions for commonly omitted concerns (e.g., rationale, alternatives, quality attributes, consequences), ensuring more comprehensive and higher quality ADRs. We envision that the labels inferred from existing decision taxonomies as well as the topic mining process (or a precomputed model) can be integrated as a way of adding metadata to the ADRs. Tools and practitioners could leverage these metadata to improve the project's AK base.

5.1 Threats to Validity

Despite the insights revealed by our empirical analysis, a number of factors may have affected the validity of our findings. Following Vidoni [18], we discuss those factors below.

Internal Validity. Our detection of ADR structure relied on conventional section headings, potentially missing custom or informal formats. Our classification of decision types and consistency relied on LLM prompting. Using LLMs offered a scalable alternative for classification, allowing us to assess section content and purpose based on textual descriptions and structured prompts. However, we acknowledge that performance can be influenced by prompt design, and LLMs may struggle with ambiguous phrasing or project-specific terminology. To mitigate these risks, two of the authors independently reviewed a subset of the LLM outputs and discussed

the identified classifications. Their assessments confirmed that the inferred labels were generally consistent with the defined categories and aligned with the interpretations throughout the study.

Construct validity. To ensure that we were measuring the intended constructs, we clearly defined our research questions and developed structured prompts based on well-scoped taxonomies for LLM-based evaluation. The classification of decision types and consistency was guided by predefined criteria rooted in established definitions. Although architectural decisions may involve multiple concerns or quality attributes simultaneously, we operationalized the classification primarily as a single-label task by assigning the most dominant category per ADR, in line with prior empirical studies. This choice enabled quantitative comparison and aggregation at scale, while acknowledging that it may abstract away secondary categories for some decisions (e.g., typically quality attributes). Nonetheless, the LLM structured response considered slots for alternative decision categories, which will be integrated into the analysis in future work.

External Validity. The dataset is limited to ADRs written in English, which may not capture documentation practices in other languages or cultural contexts. Consequently, the findings may not generalize to multi-lingual or region-specific development environments. In addition, our analysis is limited to open-source projects on GitHub, which introduces a sampling bias. This excludes private, industrial, or self-hosted repositories, as well as those on other platforms. Thus, the findings may not fully generalize to all software development environments. Overall, we believe that the scale of our dataset supports meaningful conclusions about ADR usage.

6 CONCLUSIONS AND FUTURE WORK

In this paper, we proposed an automated approach for mining and analyzing ADRs, and then applied this approach to perform an empirical study to understand concerns normally captured in ADRs in open-source projects. Our initial findings reveal the most prevalent types of design decisions, the most common topics whether architectural or not, and the use (or sometimes misuse) of the MADR template when documenting ADRs. Our analysis shows a large proportion of existence decisions, many references to technology options, deployment, infrastructure aspects and process considerations, and also a higher occurrence of quality attributes such as maintainability, security and reliability. By answering "*what*" is discussed in ADRs, this research seeks to improve AK management by identifying misalignments between the expected and actual contents of ADRs. Certainly, identifying the best ADR label for a given taxonomy is debatable for certain categories, even for human users. While still fallible, an LLM-approach seems to work better than embeddings of ADR texts, because the LLM can reason about the ADR semantics, if provided with adequate prompting and context.

Our findings open up several directions for future work. A reasonable next step is to analyze "*why*" certain documentation patterns emerge in ADRs. First, we would like to study private or industrial repositories of ADRs to assess whether the identified trends hold. Second, it would be interesting to analyze the evolution of architectural decisions over time, including correlations with project events (e.g., major refactorings, architecture stability, or project metrics). Third, the structured ADR format could be mapped to a semantic graph, capturing relationships among decisions, alternatives, rationale, and context, among others. These graphs should be useful to support AK management, generating active metadata for ADRs and enabling architects to easily search for information, identifying mismatches and quality issues. Finally, LLM-powered tools could help practitioners write (better) ADRs from basic inputs and leverage project context [12, 13].

REFERENCES

[1] R. Capilla, A. Jansen, A. Tang, P. Avgeriou, and M. A. Babar, "10 years of software architecture knowledge management: Practice and future," *Journal of Systems and Software*, vol. 116, pp. 191–205, 2016.

[2] M. Keeling, "The psychology of architecture decision records," *IEEE Software*, vol. 39, no. 6, 2022.

[3] B. Ahmeti, M. Linder, R. Groner, and R. Wohlrab, "Architecture decision records in practice: An action research study," in *European Conference on Software Architecture*. Springer, 2024, pp. 333–349.

[4] G. Buchgeher, S. Schöberl, V. Geist, B. Dorninger, P. Haindl, and R. Weinreich, "Using architecture decision records in open source projects—an msr study on github," *IEEE Access*, vol. 11, pp. 63 725–63 740, 2023.

[5] P. Kruchten, "An ontology of architectural design decisions," in *Proceedings of the 2nd Groningen Workshop on Software Variability Management*, Groningen, The Netherlands, Dec. 2004, december 3–4, 2004.

[6] O. Zimmermann, J. Koehler, F. Leymann, R. Polley, and N. Schuster, "Managing architectural decision models with dependency relations, integrity constraints, and production rules," *Journal of Systems and Software*, vol. 82, no. 8, pp. 1249–1267, 2009.

[7] P. Kherwa and P. Bansal, "Topic modeling: A comprehensive review," *EAI Endorsed Transactions on Scalable Information Systems*, vol. 7, no. 24, 7 2019.

[8] S. Gretz, A. Halfon, I. Shnayderman, O. Toledo-Ronen, A. Spector, L. Dankin, Y. Katsis, O. Arviv, Y. Katz, N. Slonim, and L. Ein-Dor, "Zero-shot topical text classification with LLMs - an experimental study," in *Findings of the Association for Computational Linguistics: EMNLP 2023*. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 9647–9676.

[9] U. Zdun, R. Capilla, H. Tran, and O. Zimmermann, "Sustainable architectural design decisions," *IEEE Software*, vol. 30, no. 6, pp. 46–53, 2013.

[10] D. Falessi, G. Cantone, and P. Kruchten, "Value-based design decision rationale documentation: Principles and empirical feasibility study," in *Seventh Working IEEE/IFIP Conference on Software Architecture (WICSA 2008)*. IEEE, 2008, pp. 189–198.

[11] R. Dhar, K. Vaidhyanathan, and V. Varma, "Can llms generate architectural design decisions? - an exploratory empirical study," pp. 79–89, 2024.

[12] R. Dhar, A. Kakran, A. Karan, K. Vaidhyanathan, and V. Varma, "Draft-ing architectural design decisions using llms," *arXiv preprint arXiv:2504.08207*, 2025.

[13] J. A. Díaz-Pace, A. Tommasel, and R. Capilla, "Helping novice architects to make quality design decisions using an llm-based assistant," in *European Conference on Software Architecture*. Springer, 2024, pp. 324–332.

[14] C. Miesbauer and R. Weinreich, "Classification of design decisions–an expert survey in practice," in *Software Architecture: 7th European Conference, ECSA 2013, Montpellier, France, July 1-5, 2013. Proceedings 7*. Springer, 2013, pp. 130–145.

[15] O. Zimmermann, J. Koehler, and F. Leymann, "Architectural decision models as micro-methodology for service-oriented analysis and design," in *SEMSOA Workshop, Hannover, Germany*, 2007.

[16] O. Zimmermann, "An architectural decision modeling framework for service-oriented architecture design," Ph.D. dissertation, Stuttgart, Univ., Diss., 2009, 2009.

[17] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed. Addison-Wesley Professional, 2012.

[18] M. Vidoni, "A systematic process for mining software repositories: Results from a systematic literature review," *Information and Software Technology*, vol. 144, p. 106791, 2022.

[19] M. Grootendorst, "Bertopic: Neural topic modeling with a class-based tf-idf procedure," *arXiv preprint arXiv:2203.05794*, 2022.

[20] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020.

[21] W. Yu, D. Iter, S. Wang, Y. Xu, M. Ju, S. Sanyal, C. Zhu, M. Zeng, and M. Jiang, "Generate rather than retrieve: Large language models are strong context generators," *arXiv:2209.10063*, 2022.

[22] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging llm-as-a-judge with mt-bench and chatbot arena," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS '23. Red Hook, NY, USA:

Curran Associates Inc., 2023.